钢铁表面缺陷检测项目（4周）

- **掌握计算机视觉和视觉大模型基础知识（2周）**
  - 大模型基础知识
    - 知识准备
      - 大模型概述与演进
        - 什么是大模型（参数量、涌现能力）
        - 发展历程（2017–2025关键节点）
        - 大模型 vs 传统深度学习模型
        - 核心能力（语言理解、上下文学习、推理、指令遵循、多模态）
      - Transformer架构详解
        - 诞生背景（RNN局限）
        - 自注意力机制（Q/K/V、缩放点积）
        - 多头注意力
        - 位置编码
        - 编码器-解码器结构
        - 前馈网络与层归一化
      - 大模型的训练范式
        - 预训练（自监督学习）
        - 监督微调（SFT）
        - 强化学习与人类反馈（RLHF）
        - 提示学习（零样本、少样本、CoT）
        - 参数高效微调（PEFT / LoRA）
      - 涌现能力与上下文学习
        - 涌现能力定义与阈值
        - 上下文学习（ICL）
        - 思维链（CoT）
        - 上下文长度
        - 知识边界与幻觉
      - 大模型评估与部署
        - 评估指标（MMLU、GSM8K、HumanEval等）
        - 推理优化（KV Cache、量化、批处理、推测解码）
        - 部署方式（API、私有化、边缘端）
        - 检索增强生成（RAG）
        - 成本分析
      - 多模态大模型
        - 定义与架构（视觉编码器、语言模型、对齐模块）
        - CLIP模型
        - 视觉问答（VQA）
        - 工业应用
      - 大模型在工业质检中的实践
        - 大模型质检 vs 传统CV质检
        - 落地挑战（精度、延迟、成本、隐私、幻觉）
        - RAG增强质检
        - 自动标注与模型迭代
        - 开源模型选型建议（Qwen、ChatGLM、DeepSeek、LLaMA等）
      - 总结与展望
        - 核心概念回顾
        - 对工业质检的影响
        - 未来趋势（轻量化、推理加速、多模态融合、持续学习、可信AI）
    - 任务实施
      - 系列小实训
  - 计算机视觉与工业质检基础
    - 知识准备
      - 人工智能与计算机视觉概览
        - AI发展简史
        - 机器学习 vs 深度学习
        - 计算机视觉定义与目标
        - 主要任务层级（底层、中层、高层）
        - 图像分类、目标检测、图像分割
        - 其他CV任务（跟踪、姿态估计、生成、多模态检索）
        - 工业视觉系统组成
        - 工业质检与CV结合点
        - 工业相机选型、RTSP协议、OpenCV读取、实时检测集成
      - 工业质检深度剖析
        - 传统人工质检痛点（速度慢、主观、疲劳、成本高）
        - 具体困境数据（漏检率、节拍、流动率、培训周期）
        - 自动化质检演进路径（基于规则→传统ML→深度学习→大模型+决策）
        - 独特挑战（小目标、类间相似、类内差异、数据不均衡、实时性、环境干扰）
        - 典型缺陷类型（钢铁、3C、汽车）
        - 评估指标（漏检率、过杀率）
        - 缺陷严重程度分级
        - 部署模式（边缘、云端、混合）
        - 质检数据管理与价值
        - 行业现状与趋势
      - 目标检测核心概念与数据集
        - 边界框表示法（绝对、中心尺寸、归一化）
        - 交并比（IoU、GIoU、DIoU、CIoU）
        - 置信度含义
        - 非极大值抑制（NMS）
        - 精确率与召回率
        - PR曲线与AP、mAP
        - 数据集格式（VOC、COCO、YOLO）
        - 标注规范与质量控制
        - 训练/验证/测试集划分
        - 数据增强策略
      - YOLO系列算法详解
        - 核心理念（You Only Look Once）
        - YOLOv1结构与局限
        - YOLOv2改进（Anchor、BN、高分辨率、多尺度）
        - YOLOv3飞跃（多尺度预测、残差、Darknet-53）
        - YOLOv4与v5工程优化
        - YOLOv8突破（无锚框、解耦头、任务对齐、数据增强）
        - YOLOv10与v12最新进展
        - 选型考虑因素
        - 在工业质检中的优势与局限性
      - 模型评估与优化实战
        - 过拟合与欠拟合
        - 超参数调优（学习率、批量大小、优化器、权重衰减）
        - 混淆矩阵分析
        - 迁移学习与微调
        - 模型轻量化技术（剪枝、量化、知识蒸馏）
    - 任务实施
      - 系列小实训
  - 视觉大模型与零样本目标检测
    - 知识准备
      - 视觉大模型（VLM）基础
        - 从单模态到多模态
        - 大模型定义与特征
        - 主要代表（CLIP、Flamingo、LLaVA、Qwen-VL、GPT-4V）
        - VLM模型架构（视觉编码器、语言模型、对齐模块）
        - 视觉Transformer（ViT）
        - 训练数据（图文对、VQA、检测定位、指令数据）
        - 零样本能力来源
        - 零样本目标检测（ZSD）原理、流程、典型模型
        - 准确度分析、推理成本与延迟
        - 幻觉问题
        - 安全性与隐私问题
      - Qwen-VL深度解析
        - 通义千问视觉系列（基础版、Plus、Max、Qwen3-VL）
        - 核心能力（图像描述、VQA、目标定位、OCR）
        - 输入输出格式
        - API调用流程
        - 稳定输出边界框技巧
        - Prompt工程进阶（角色扮演、任务分解、思维链、负向约束）
        - 检测结果可靠性校验
        - 与其他VLM对比
      - 零样本目标检测深入
        - 零样本学习基本范式
        - 检测难点（定位、背景抑制、类别混淆）
        - 基于VLM的检测流程（两阶段、端到端）
        - 评估协议（mAP_seen, mAP_unseen, 调和平均）
        - 典型应用（新缺陷响应、罕见缺陷、辅助标注、跨领域）
        - 局限性与改进
        - 少样本检测（Few-shot）
        - 开放词汇检测（Open-Vocabulary）
      - 工业场景中的VLM应用
        - VLM在质检中的定位（辅助、补充、复核）
        - 辅助数据标注
        - 缺陷根因分析
        - 生成质检报告
        - 边缘部署可行性
        - 成本效益分析
        - 未来发展影响
      - 大模型落地挑战与最佳实践
        - 延迟与吞吐量
        - 成本控制策略
        - 数据隐私保护
        - 结果不稳定的应对（多次采样、规则后处理）
        - 持续优化机制（收集Bad Case、双轨优化）
        - 可解释性（热力图、全链路追溯）
        - 模型版本管理与回滚
        - 合规与伦理
        - 团队技能要求
        - VLM+YOLO双引擎模式总结
    - 任务实施
      - 系列小实训
  - 数据工程与模型训练实践
    - 知识准备
      - 数据标注基础
        - 什么是数据标注
        - 常见标注类型（图像、文本、表格）
        - 图像标注详解（2D框、旋转框、多边形、关键点等）
        - 标注工具选型（LabelImg、CVAT、商业平台、人机协同）
        - 质量控制（一致性、指南、分层抽查、仲裁）
        - 标注流程与角色（SOP、KPI）
        - 常见错误与应对
        - 标注格式与导出（COCO、VOC、YOLO）
        - 成本估算
      - 数据工程基础
        - 数据来源与获取（公开、自采、合成）
        - 数据质量初筛
        - 标签与类别管理（统一命名、连续ID）
        - 标注格式转换（VOC↔COCO↔YOLO）
        - 数据集划分策略（分层采样）
        - 数据目录结构规范
        - 数据统计报告
        - 探索性数据分析（EDA）——图像、表格、文本
        - EDA输出与行动指南
        - 数据增强（几何、颜色、噪声、高级融合）
        - 数据版本管理
      - 模型训练基础
        - 训练环境准备（硬件、软件、预训练权重）
        - 核心超参数（Epochs、Batch Size、Learning Rate、Optimizer、ImgSz）
        - 训练过程监控（损失曲线、指标、权重备份）
        - 早停（Early Stopping）
        - 类别不平衡处理（类别权重、重采样、增强）
        - 训练失败常见原因与解决
        - 模型评估基础（分类、检测、分割指标）
        - 错误分析（漏检、误检、分类错误）
        - 置信度阈值调优
        - 特征工程（数值、类别、缺失值、特征筛选）
        - 模型可解释性简介（LIME、SHAP）
        - 模型对比与选择
        - 迭代改进闭环、实验记录模板
      - 模型部署与推理优化
        - 模型轻量化技术（量化、剪枝、知识蒸馏、轻量架构）
      - 大模型API调用基础
        - API Key获取与安全配置
        - 请求构造与响应解析（图像编码、System Prompt、温度参数、容错重试）
        - 成本控制策略（预算上限、混合架构、缓存、性价比模型）
    - 任务实施
      - 系列小实训
  - 模型优化与部署进阶
    - 知识准备
      - 模型可解释性 — Grad-CAM
        - 为什么需要可解释性
        - Grad-CAM原理
        - 在YOLO上实现Grad-CAM
        - 结果分析（正确关注、错误关注）
      - 模型轻量化 — 剪枝与蒸馏
        - 轻量化概述
        - 通道剪枝原理与流程
        - 使用torch_pruning剪枝ResNet
        - 知识蒸馏原理
        - Ultralytics官方蒸馏API
      - 模型量化与TensorRT加速
        - 模型量化概述（FP32、FP16、INT8）
        - ONNX导出与推理
        - TensorRT加速原理与部署流程
      - 模型推理与结果可视化
        - 检测结果可视化要求
        - 批量推理与性能统计
        - 可视化监控面板
      - 缺陷度量与统计分析
        - 缺陷几何度量（面积、长宽比、物理尺寸转换）
        - 统计分析报告生成（特征提取、可视化、报告导出）
        - 工艺参数关联挖掘
    - 任务实施
      - 系列小实训
  - OpenClaw框架技术解析
    - 知识准备
      - 原理与架构
        - 什么是OpenClaw（NL2Sys、模块化、Agent驱动）
        - 起源与社区（龙虾梗）
        - 核心设计哲学（本地优先、纯文本配置、透明可控、长期记忆）
        - 架构分层（交互层、网关层、智能体层、执行层）
        - 核心原理：NL2Sys与"大脑/躯干"解耦
        - 消息处理完整流程（10步）
        - ReAct推理循环（思考-行动-观察）
      - 核心组件详解
        - 核心组件概览（Gateway、Agent、Skill、Provider）
        - Gateway详解（统一入口、认证鉴权、智能路由）
        - Agent工作原理
        - Provider与模型选择
        - Channels（渠道层）详解
      - Skill开发与实践
        - Skill定义与结构（目录、SKILL.md、脚本、环境变量）
        - 如何注册与发现Skill（自动注册、语义匹配）
        - 权限与安全声明（沙箱隔离）
        - 配置文件（openclaw.json）
        - 部署方式（一键脚本、npm、Docker、配置向导）
      - 高级特性与系统设计
        - 四层记忆系统（短期、近端、长期MEMORY.md、人格SOUL.md）
        - Sub-Agent与并行执行
        - 多层容错机制（Key轮换、模型降级、上下文压缩、重试、心跳）
        - 安全体系（沙箱隔离、设备白名单、审计日志、恶意代码扫描）
        - 部署模式（Local、Remote、Hybrid）
      - 生态、对比与应用
        - OpenClaw生态体系（ClawHub、腾讯SkillHub、大厂案例）
        - 与其他AI Agent框架对比（LangChain、AutoGen、Dify）
        - 典型使用场景与企业案例
        - 常见问题排查与最佳实践
        - 项目需求分析：YOLO+VLM双Skill设计
        - 总结：为什么选择OpenClaw
    - 任务实施
      - 系列小实训
- **开发钢铁表面缺陷检测系统（2周）**
  - 项目现状分析与业务痛点
    - 企业背景与项目动因
    - 当前检测流程（产线速度、人工目视、抽检比例、纸质记录）
    - 历史缺陷数据分析（客诉、缺陷类型分布、经济损失）
    - 痛点1：速度与节拍不匹配
    - 痛点2：主观性与一致性差
    - 痛点3：新缺陷适应慢
    - 痛点4：数据孤岛与无法追溯
    - 项目价值主张
  - 需求分析与系统目标
    - 功能需求（实时检测、双引擎、人工审核、统计报表、数据导出）
    - 非功能需求（性能、准确率、可用性、安全性、可扩展性）
    - 用户角色分析（质检员、主管、AI工程师、工艺工程师）
    - 用例图（质检员、主管、工程师流程）
    - 系统边界与外部接口
    - 关键性能指标（KPI）
    - 约束条件（硬件、网络、标注数据）
    - 风险识别与应对
  - 系统架构与设计方案
    - 总体架构图（展示层、业务层、能力层、数据层、基础设施）
    - 数据流设计（检测、审核、导出）
    - 模块划分与职责（app.py, detection_engine.py, db_manager.py等）
    - 数据库设计（inspection_records表）
    - 双引擎协同架构（YOLO直出、VLM复核、人工兜底）
    - 部署架构（单机、云边混合、弹性扩展）
    - 安全设计（数据隐私、凭证隔离、加密）
    - 可扩展性设计（新增引擎、导出格式、Skill）
    - 容错与恢复
    - 技术选型理由（Gradio、SQLite、Ultralytics、OpenClaw）
  - 系统开发
    - 开发环境与工具链
      - 硬件环境（工控机、RTX 4060 8G、工业相机）
      - 软件环境（Windows 10/11、Python 3.8+、CUDA 11.x）
      - 核心依赖库（PyTorch、Ultralytics、OpenCV、Gradio、sqlite3）
      - 版本控制与团队协作（Git、GitLab）
    - 项目代码结构
      - 目录组织（data/、models/、utils/、skills/、tests/、scripts/）
      - 配置管理（config.yaml、环境变量、模型路径）
      - 主入口（main.py、app.py、cli.py）
    - 核心模块实现
      - 图像采集模块
        - RTSP流读取与帧缓存（OpenCV、多线程缓冲队列）
        - 相机硬触发集成（PLC信号交互）
      - 检测引擎模块
        - YOLO检测器封装（模型加载、预处理、推理、后处理）
        - VLM检测器封装（API调用、重试机制、结果解析）
        - 双引擎统一接口 detect(image, engine, **kwargs)
      - 数据持久化模块
        - SQLite表结构创建与迁移
        - 异步写入与批量插入优化
        - 历史查询与统计接口
      - Web可视化工作台
        - 实时检测页面（视频流/图片上传、结果展示）
        - 审核页面（原图+预测框、修正表单、提交）
        - 报表页面（图表、日期筛选、导出按钮）
      - 导出模块
        - CSV导出（字段映射、编码处理）
        - Bad Case数据集导出（图片复制、YOLO标签生成）
        - HTML报告生成（模板渲染、图表嵌入）
    - OpenClaw Skill 开发实现
      - yolo-detect Skill
        - SKILL.md 编写（名称、描述、权限、输入输出示例）
        - detect.py 实现（调用 detection_engine.YOLODetector）
        - 本地测试与调试
      - vlm-detect Skill
        - SKILL.md 编写（描述调用VLM API的能力）
        - detect.py 实现（异步调用、超时处理、JSON解析）
        - 安全沙箱配置（仅允许 network 权限）
      - 通用检测基类
        - BaseDetector 与代码复用
    - Agent 与 Gateway 配置
      - openclaw.json 配置示例（provider、agent、skill注册）
      - Gateway 启动命令与自检
      - 自然语言意图测试（"检测这张钢板的表面缺陷"）
    - 自动化脚本与工具
      - 模型训练脚本（train_yolo.py：支持YOLOv8/v10/v12）
      - 数据集准备脚本（prepare_dataset.py：格式转换、划分）
      - 性能基准测试脚本（benchmark.py：延迟、吞吐、GPU利用率）
      - 日志收集与分析脚本（log_analyzer.py）
    - 开发过程中的问题与解决记录
      - 问题1：Gradio 与 OpenClaw 端口冲突 → 修改默认端口分离
      - 问题2：VLM API 返回格式不稳定 → 增加 JSON Schema 校验与容错
      - 问题3：SQLite 并发写入锁 → 使用连接池与写队列
      - 问题4：YOLO小目标漏检 → 提升输入分辨率并微调 anchor
    - 持续集成与交付
      - 自动化测试流水线（单元测试、集成测试）
      - 模型版本打包（模型文件 + 标签映射 + 配置文件）
      - 一键部署脚本（setup.bat / install.sh）
      - 更新机制（增量模型替换、灰度发布脚本）
  - OpenClaw集成设计
    - 引入OpenClaw的原因
    - 两个Skill的设计（yolo-detect、vlm-detect）
    - Agent设计（detect-analyst）
    - Gateway配置要点
    - 自然语言接口的好处
    - 与Gradio工作台的关系（互补、过渡、未来）
    - 实施步骤
  - 系统测试与上线部署
    - 单元测试策略
    - 集成测试
    - 性能测试
    - 用户验收测试（UAT）
    - 部署方案（工控机、服务化、开机自启）
    - 监控与告警
    - 文档与培训

Presented with xmind